

Segundo Cérebro — Explicação Completa do Projeto

Guia técnico gerado para consulta

Segundo Cérebro — Guia de Explicação do Código

Este documento explica, arquivo por arquivo, o que cada parte do projeto **Segundo Cérebro** faz, na ordem que facilita o entendimento: da base visual até a tela mais complexa do app.

1. `src/theme.js` — a base visual

Esse arquivo só exporta três objetos JavaScript simples: `colors`, `spacing` e `radius`. Não tem lógica nenhuma, é puro dado, e é importado por praticamente todo componente do app.

`colors` define a paleta:

- `bg`: fundo geral do app (quase preto azulado)
- `surface`: fundo dos cards/caixas
- `surfaceAlt`: fundo um pouco mais claro, usado dentro de inputs e blocos de código
- `border`: cor das bordas finas usadas em quase todo componente
- `accent`: azul usado em botões principais e links de destaque
- `accentDim`: variação mais escura do accent, usada como cor padrão de avatar quando o usuário não escolhe nenhuma
- `textPrimary`, `textSecondary`, `textMuted`: três níveis de “intensidade” de texto, criando hierarquia visual sem precisar variar o tamanho da fonte
- `danger` e `success`: cores semânticas (vermelho/verde), ainda não usadas no código atual — reservadas para uso futuro
- `link`: azul-claro usado especificamente nos wikilinks `[[...]]`

`spacing` é uma escala de espaçamento (4, 8, 16, 24, 32) que substitui números mágicos espalhados pelo código. Em vez de `padding: 16`, o código escreve `padding: spacing.md`, mantendo tudo consistente.

`radius` segue o mesmo princípio, mas para bordas arredondadas (6, 10, 16).

2. `src/utils/armazenamento.js` — a camada de banco de dados

Único lugar do projeto que conversa com o SQLite. Nenhum componente ou tela acessa o banco diretamente — todos passam por aqui (camada de acesso a dados).

Conexão com o banco

```
let db = null;

async function getDb() {
  if (db) return db;
  db = await SQLite.openDatabaseAsync('segundo_cerebro.db');
  return db;
}
```

`db` é uma variável de módulo (existe uma única vez, compartilhada pelo app inteiro). `getDb()` é um padrão de cache: na primeira chamada, abre o arquivo `segundo_cerebro.db`; nas seguintes, devolve a conexão já aberta.

Criação das tabelas

```
async function inicializarBanco() {
  const banco = await getDb();
  await banco.execAsync(`
    PRAGMA journal_mode = WAL;
    CREATE TABLE IF NOT EXISTS cerebros (...)
    CREATE TABLE IF NOT EXISTS notas (...)
  `);
}
```

Roda uma vez quando o app abre. `PRAGMA journal_mode = WAL` melhora performance e permite leitura/escrita simultânea com segurança. As tabelas só são criadas se ainda não existirem (`IF NOT EXISTS`).

- `cerebros`: `id`, `name`, `description`, `color`, `criado_em`
- `notas`: `id`, `cerebro_id` (chave estrangeira), `title`, `content`, `preview`, `updated_at`

O `FOREIGN KEY ... ON DELETE CASCADE` é o detalhe importante: ao deletar um cérebro, todas as notas filhas são apagadas automaticamente pelo banco, sem código extra.

Funções CRUD

Todas seguem o mesmo padrão: try/catch + log de erro + retorno seguro em caso de falha, para nunca quebrar a interface.

- `getCerebros()` — busca todos os cérebros ordenados do mais antigo pro mais novo.
- `salvarCerebro(cerebro)` — usa `INSERT ... ON CONFLICT(id) DO UPDATE` (upsert): se o `id` já existe, atualiza; senão, insere. Permite usar a mesma função tanto para criar quanto editar.
- `deletarCerebro(cerebroId)` — deleta pelo `id`. O `?` no SQL é um placeholder protegido contra SQL injection.
- `getNota(cerebroId, notaId)` — busca uma nota específica (`getFirstAsync` pega só a primeira linha).
- `salvarNota(cerebroId, nota)` — mesmo padrão de upsert.
- `getListaNotas(cerebroId)` — pega as notas de um cérebro, mas só campos leves (`id`, `title`, `preview`, `updated_at`), sem o `content` inteiro — usada para montar listas, não precisa do texto completo.
- `deletarNota(cerebroId, notaId)` — deleta uma nota específica.

Geração de ID

```
function gerarId() {  
  return Date.now().toString(36) + Math.random().toString(36).slice(2, 7);  
}
```

Combina o timestamp atual (em base 36) com 5 caracteres aleatórios. Não é um UUID de verdade, mas é suficiente para um app local sem sincronização entre dispositivos.

3. `src/utils/markdownUtil.js` — o parser de Markdown caseiro

Não usa nenhuma biblioteca de Markdown — é um parser simplificado escrito à mão com expressões regulares (regex).

`extrairWikiLinks(texto)`

```
const regex = /\[([^\]]+)\]/g;
```

Procura por `[[qualquer coisa que não seja ]]`. O `g` faz a regex encontrar todas as ocorrências. `match[1]` é o conteúdo capturado dentro dos colchetes (o nome do link). No final, `[...new Set(links)]` remove duplicatas.

tokenizarLinha(linha)

```
const padrao = /(\\\[^\]]+\]|\\*\\\[^\]]+\\*\\)|\\*\\\[^\]]+\\*\\)|(`[^\`]+`)/g;
```

A parte mais densa do projeto. A regex tem 4 alternativas: wikilink, negrito, itálico ou código. A função varre a linha procurando essas marcações; o que está **entre** marcações vira um token `'text'`; o que é capturado vira `link`, `bold`, `italic` ou `code` (removendo os símbolos via `.slice()`). O resultado é uma lista de tokens usada pelo `PreviewMarkdown.js` para renderizar cada pedaço com o estilo certo.

classificarLinha(linha)

```
if (linha.startsWith('# ')) return { type: 'h1', content: linha.slice(2) };
```

Olha para o **início** da linha inteira e decide o tipo estrutural: título nível 1/2/3, item de lista, citação, linha divisória, linha em branco, ou parágrafo comum (padrão).

4. App.js — o ponto de entrada

```
const Pilha = createNativeStackNavigator();
```

Cria um navegador em pilha: cada tela nova “empilha” sobre a anterior, e o botão voltar desempilha.

Inicialização do banco

```
const [bancoPronto, setBancoPronto] = useState(false);

useEffect(() => {
  inicializarBanco()
    .then(() => setBancoPronto(true))
    .catch(e => { setBancoPronto(true); });
}, []);
```

`useEffect` com array vazio `[]` roda uma única vez, na montagem. Chama a função que cria as tabelas e só depois libera a navegação. Mesmo com erro, segue em frente — o app tenta funcionar em vez de travar numa tela de erro. Enquanto `bancoPronto` é `false`, mostra apenas um spinner centralizado.

Estrutura de navegação

`SafeAreaProvider` (áreas seguras de notch) → `NavigationContainer` → `Pilha.Navigator`
com três telas:

- **Início**: sem header próprio (`headerShown: false`), pois a `TelaInicial.js` desenha seu próprio cabeçalho.
- **Cerebro**: título dinâmico, pega o nome do cérebro vindo via `route.params`.
- **Editor**: sem título, botão de voltar escrito “Notas”.

5. `src/components/CartaoCerebro.js` — cartão de vault na tela inicial

```
const iniciais = cerebro.name
  .split(' ')
  .slice(0, 2)
  .map(w => w[0]?.toUpperCase() ?? '')
  .join('');
```

Pega o nome, separa em palavras, pega as 2 primeiras, extrai a primeira letra maiúscula de cada uma e junta. Ex: “Estudos de Física” → “ED”. O `?.` e `?? ''` protegem contra nome vazio ou estranho.

O JSX: `TouchableOpacity` (toque curto abre, toque longo abre menu de editar/deletar) com avatar colorido + iniciais, nome + metadado (`X notas · descrição`), e uma seta decorativa `>`.

6. `src/components/ItemNota.js` — item de nota na lista

```
function formatarData(isoString) {
  const dias = Math.floor(diff / (1000 * 60 * 60 * 24));
  if (dias === 0) return 'hoje';
  if (dias === 1) return 'ontem';
  if (dias < 7) return `${dias} dias atrás`;
  return d.toLocaleDateString('pt-BR', { day: '2-digit', month: 'short' });
}
```

Calcula a diferença em dias entre agora e a data da nota, e retorna um texto amigável (“hoje”, “ontem”, “N dias atrás” ou a data formatada se passou mais de uma semana).

O componente mostra: título (1 linha), data ao lado, preview do conteúdo (até 2 linhas), se existir.

7. `src/components/ModalNovoCerebro.js` — modal de criar/editar vault

```
const PALETA = ['#3a5cbf', '#5b8af5', ...];
```

Lista fixa de cores disponíveis para o avatar do cérebro.

Lógica de criar vs. editar

```
useEffect(() => {
  if (dadosIniciais) {
    setNome(dadosIniciais.name ?? '');
    // ...
  } else {
    setNome('');
    // ...
  }
}, [visivel]);
```

O componente serve tanto para criar quanto editar, dependendo se recebe `dadosIniciais` (cérebro existente) ou não. Toda vez que `visivel` muda (modal abre/fecha), o effect roda: se tem `dadosIniciais`, preenche os campos para edição; senão, limpa tudo para um formulário em branco.

Validação

```
function handleConfirmar() {
  const nomeTrimado = nome.trim();
  if (!nomeTrimado) return;
  aoConfirmar({ name: nomeTrimado, description: descricao.trim(), color:
corSelecionada });
}
```

Nome não pode ser vazio. Se válido, chama `aoConfirmar` (recebida via props) — quem decide o que fazer com os dados é o componente pai, não o modal.

No JSX: `Modal` nativo com fundo escurecido, `KeyboardAvoidingView` (ergue conteúdo quando o teclado abre no iOS), campos de nome (obrigatório) e descrição (opcional), bolinhas coloridas selecionáveis (a ativa ganha borda extra), e botões Cancelar / Criar-Salvar (desabilitado se o nome estiver vazio).

8. `src/screens/TelaInicial.js` — tela de lista de vaults

Recarregamento ao focar

```
useFocusEffect(  
  useCallback(() => {  
    carregarCerebros();  
  }, [])  
);
```

Diferente do `useEffect`, o `useFocusEffect` roda toda vez que a tela **ganha foco** (volta de outra tela navegando). Garante que a contagem de notas atualize ao voltar de outro vault.

Enriquecimento de dados

```
async function carregarCerebros() {  
  const lista = await getCerebros();  
  const enriquecida = await Promise.all(  
    lista.map(async c => {  
      const notas = await getListaNotas(c.id);  
      return { ...c, noteCount: notas.length };  
    })  
  );  
  setCerebros(enriquecida);  
}
```

Busca os cérebros e, para cada um, busca as notas só para contar (`Promise.all` roda em paralelo, mais rápido que sequencial). O resultado ganha um campo `noteCount` calculado em memória, que não existe no banco.

Criar vs. editar

```
async function handleCriar({ name, description, color }) { /* monta objeto novo  
com id e data */ }  
async function handleEditar({ name, description, color }) { /* mantém id e  
criadoEm originais */ }
```

Duas funções passadas ao modal dependendo do contexto: `aoConfirmar={cerebroEditando ? handleEditar : handleCriar}`. `handleEditar` usa spread (`{...cerebroEditando, name, description, color}`) para preservar campos não editados.

Menu de ações

```
function handlePressionarLongo(cerebro) {
  Alert.alert(cerebro.name, 'O que você quer fazer?', [
    { text: 'Editar', onPress: () => { /* ... */ } },
    { text: 'Deletar', style: 'destructive', onPress: () =>
confirmarDelecao(cerebro) },
    { text: 'Cancelar', style: 'cancel' },
  ]);
}
```

Usa `Alert` nativo como menu ao segurar um cartão. `confirmarDelecao` é uma segunda confirmação antes de deletar (boa prática para ações destrutivas).

O JSX: cabeçalho customizado (nome do app + contagem de vaults + botão “+Novo”), `FlatList` de `CartaoCerebro` (com estado vazio customizado), e o `ModalNovoCerebro` controlado pelos states `modalVisivel` e `cerebroEditando`.

9. `src/screens/TelaCerebro.js` — lista de notas de um vault

```
const { cerebro } = route.params;
```

Recebe o objeto `cerebro` inteiro via parâmetro de navegação.

Filtro de busca

```
function notasFiltradas() {
  if (!busca.trim()) return notas;
  const q = busca.toLowerCase();
  return notas.filter(n => n.title?.toLowerCase().includes(q));
}
```

Filtro em memória (não consulta o banco de novo): retorna tudo se a busca estiver vazia, ou filtra por título (case-insensitive).

Abrir/criar nota

```
function abrirNota(notaId) {
  navigation.navigate('Editor', {
    cerebroId: cerebro.id,
```



```

        notaId: notaId ?? gerarId(),
        ehNova: !notaId,
        nomeCerebro: cerebro.name,
    });
}

```

Usada tanto para abrir nota existente quanto para criar nova. Se `notaId` é passado, usa ele e `ehNova = false`; se é `null` (botão “+”), gera id novo na hora e marca `ehNova = true`.

O resto segue o padrão da `TelaInicial`: busca, `FlatList` de `ItemNota`, menu de ações ao segurar, confirmação de delete, e botão flutuante circular para nova nota.

10. `src/screens/TelaEditor.js` — a tela mais complexa do app

```

const { cerebroId, notaId, ehNova, initialTitle } = route.params;
const refInput = useRef(null);
const refSelecao = useRef({ start: 0, end: 0 });

```

`refInput` referencia diretamente o `TextInput` do editor, para chamar `.focus()` programaticamente. `refSelecao` guarda a posição do cursor — usar `useRef` (em vez de `useState`) evita re-renderizações a cada movimento de cursor.

Cabeçalho dinâmico

```

useLayoutEffect(() => {
  navigation.setOptions({
    title: '',
    headerRight: () => ( /* botões Preview/Editar e Salvar */ ),
  });
}, [modo, titulo, conteudo]);

```

`useLayoutEffect` roda de forma síncrona antes da tela aparecer, evitando “piscar” um header errado. Sempre que `modo`, `titulo` ou `conteudo` mudam, o header é redesenhado, garantindo que o botão Salvar sempre veja os valores atuais.

Carregamento condicional

```

async function carregarNota() {
  if (ehNova) return;
  const nota = await getNota(cerebroId, notaId);
}

```

```
// ...  
}
```

Só busca a nota no banco se **não** for uma nota nova — uma nota nova ainda não existe no banco.

Salvar nota

```
async function handleSalvar() {  
  const nota = {  
    id: notaId,  
    title: titulo.trim() || 'Sem título',  
    content: conteudo,  
    preview: conteudo.slice(0, 120).replace(/[#*`>\\[\]]/g, '').trim(),  
    updatedAt: new Date().toISOString(),  
  };  
  await salvarNota(cerebroId, nota);  
}
```

O `preview` pega os primeiros 120 caracteres e remove símbolos de Markdown via regex, gerando um resumo limpo para exibir nas listas.

Barra de ferramentas Markdown

```
function inserirMarkdown(btn) {  
  const { start, end } = refSelecao.current;  
  const selecionado = conteudo.slice(start, end);  
  // ...  
  setConteudo(novoTexto);  
  setTimeout(() => { refInput.current?.focus(); }, 50);  
}
```

`BOTÕES_BARRA` define cada botão: H1/H2 inserem prefixo sem envolver nada (`envolver: false`); Negrito/Itálico/Código **envolvem** o texto selecionado com o símbolo nas duas pontas (`envolver: true`); o botão `[]` insere `[]` antes e `]]` depois via `sufixo`. A lógica recorta o texto em três partes (antes, selecionado, depois) e remonta com os símbolos no meio. O `setTimeout` de 50ms garante que o teclado continue aberto após a mudança de texto.

Sistema de wikilinks

```
async function handleClicarLink(tituloLink) {  
  const existente = notasCerebro.find(n => n.title?.toLowerCase() ===  
    tituloLink.toLowerCase());
```

```

    if (existente) {
      navigation.push('Editor', { cerebroId, notaId: existente.id, ehNova:
false });
    } else {
      Alert.alert(`${tituloLink}`, 'Essa nota não existe ainda. Criar agora?', [
        { text: 'Cancelar', style: 'cancel' },
        { text: 'Criar nota', onPress: () => {
          navigation.push('Editor', { cerebroId, notaId: gerarId(), ehNova: true,
initialTitle: tituloLink });
        }
      ]);
    }
  }
}

```

O coração do sistema de links entre notas. Procura entre as notas carregadas do vault se existe alguma com aquele título exato (ignorando maiúsculas/minúsculas). Se existe, navega com `navigation.push` (sempre empilha uma tela nova, mesmo repetida — permite cadeias do tipo Nota A → Nota B → Nota A). Se não existe, pergunta se quer criar uma nova nota já com o título pré-preenchido.

Links quebrados

```

const wikiLinks = extrairWikiLinks(conteudo);
const titulosExistentes = notasCerebro.map(n => n.title);
const linkQuebrado = wikiLinks.filter(l => !titulosExistentes.includes(l));

```

Recalculado a cada renderização: quais links existem no texto, quais notas existem no vault, e quais links não batem com nenhuma nota (links quebrados) — usado para colorir os “chips” no rodapé de forma diferente.

No JSX: input de título sempre visível, alterna entre modo editor (barra de ferramentas + `TextInput` multilinha monoespaçado) e modo preview (`PreviewMarkdown`), e rodapé com chips clicáveis para cada link (azul = existe, cinza tracejado = não existe).

11. `src/components/PreviewMarkdown.js` — renderiza o Markdown

Tokens inline

```

function TokensInline({ tokens, aoClicarLink }) {
  return (

```

```

    <Text>
      {tokens.map((token, i) => {
        if (token.type === 'link') {
          return <Text key={i} style={styles.link} onPress={() => aoClicarLink?.
(token.value)}>{token.value}</Text>;
        }
        // ...
      })}
    </Text>
  );
}

```

Recebe a lista de tokens gerada por `tokenizarLinha` e transforma cada um num `<Text>` aninhado com o estilo certo. Em React Native, `<Text>` dentro de `<Text>` é a forma correta de misturar estilos numa mesma “linha” (equivalente a `<span>`s dentro de um parágrafo HTML). O link recebe `onPress`, repassado até o `handleClicarLink` da `TelaEditor.js`.

Renderização linha a linha

```

export default function PreviewMarkdown({ conteudo, aoClicarLink }) {
  const linhas = (conteudo ?? '').split('\n');
  // para cada linha: classificarLinha + tokenizarLinha
}

```

Quebra o conteúdo em linhas e aplica, para cada uma, primeiro `classificarLinha` (estrutura: título, lista, citação...) e depois `tokenizarLinha` no conteúdo sem o prefixo estrutural (formatação inline: negrito, itálico, link, código). É um parser em duas camadas — por isso `[[link]]` funciona dentro de um item de lista ou citação.

O restante é um conjunto de condicionais visuais: linha em branco vira espaço, `hr` vira barra divisória, `quote` vira bloco com borda lateral colorida, `li` vira linha com marcador “•”, e os demais tipos (`h1`, `h2`, `h3`, `p`) escolhem o estilo de texto correspondente.

Resumo da arquitetura geral

Camada	Responsabilidade
<code>theme.js</code>	Define a aparência (cores, espaçamentos, bordas)
<code>armazenamento.js</code>	Única porta de acesso ao banco SQLite

Camada	Responsabilidade
<code>markdownUtil.js</code>	Parser de texto (estrutura + formatação inline)
<code>components/</code>	Blocos visuais reutilizáveis e “burros” — recebem dados via props
<code>screens/</code>	Donas da lógica — buscam dados, decidem ações, passam dados prontos aos componentes
<code>App.js</code>	Amarra a navegação e garante que o banco esteja pronto

O fluxo típico é: uma **tela** chama uma função de `armazenamento.js` para ler/escrever dados, guarda o resultado em `state`, e repassa esse estado para **componentes** visuais via props. Ações do usuário (toques, digitação) disparam funções na tela, que atualizam o `state` e, se necessário, persistem no banco novamente.